

WebApi Handson Solution

1. WebApi Hands-on Solution

Objectives Addressed

- * **RESTful Web Service, Web API & Microservice:** REST (Representational State Transfer) is an architectural style for providing standards between computer systems on the web. Web API is an application programming interface for either a web server or a web browser. Microservices are a software architecture style in which applications are composed of small, independent modules that communicate with each other using well-defined APIs.
- * **Difference between WebService & WebAPI:** Web Services are typically SOAP-based and return XML, whereas Web API is REST-based and can return XML, JSON, or any other format.
- * **HttpRequest & HttpResponse:** HttpRequest encapsulates the details of a client request (headers, body, URL), and HttpResponse encapsulates the server's response (status code, body, headers).
- * **Action Verbs:** `HttpGet` (read data), `HttpPost` (create data), `HttpPut` (update data), `HttpDelete` (delete data).
- * **HttpStatusCodes:** 200 OK (success), 500 InternalServerError (server failure), 401 Unauthorized (authentication needed), 400 BadRequest (invalid client request).
- * **Structure:** Controllers inherit from `ControllerBase` (or `ApiController` in .NET Framework).

Solution Implementation

Below is the code for creating a simple .NET Core Web API with a controller containing read and write actions.

1. ValuesController.cs

```
```csharp
using Microsoft.AspNetCore.Mvc;
using System.Collections.Generic;

namespace WebApi_Handson.Controllers
{
 [Route("api/[controller]")]
 [ApiController]
 public class ValuesController : ControllerBase
```

## WebApi Handson Solution

```
{
 // GET api/values
 [HttpGet]
 public ActionResult<IEnumerable<string>> Get()
 {
 return new string[] { "value1", "value2" };
 }

 // GET api/values/5
 [HttpGet("{id}")]
 public ActionResult<string> Get(int id)
 {
 return "value";
 }

 // POST api/values
 [HttpPost]
 public void Post([FromBody] string value)
 {
 }

 // PUT api/values/5
 [HttpPut("{id}")]
 public void Put(int id, [FromBody] string value)
 {
 }

 // DELETE api/values/5
 [HttpDelete("{id}")]
 public void Delete(int id)
 {
 }
}
}
```

# WebApi Handson Solution

## ### 2. Startup.cs / Program.cs Configurations

In .NET Core, `Startup.cs` (or `Program.cs` in .NET 6+) handles dependency injection and request pipeline configurations.

```
```csharp
// Example configuration in Program.cs (.NET 6+)
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();

var app = builder.Build();

// Configure the HTTP request pipeline.
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```
```

## ## Testing the API

Upon running the application, navigate to `/api/values` in your browser.

**\*\*Expected Output (JSON):\*\***

```
```json
[
  "value1",
  "value2"
]
```
```

## WebApi Handson Solution

This confirms that the GET action method result is returned as expected.